

REPORTE DE CALIDAD DE SOFTWARE

Análisis Estático · Métricas de Calidad · Complejidad Ciclomática

MedTriage – Sistema de Gestión de Turnos y Triage Médico Automatizado

1. Introducción

El presente documento constituye el Reporte de Calidad de Software del sistema MedTriage. Al tratarse de un proyecto académico en etapa de documentación — sin código fuente implementado aún — el reporte adopta el enfoque de análisis estático proyectado: define la configuración de la herramienta SonarQube, establece los umbrales de calidad (Quality Gates), proyecta las métricas esperadas con base en la arquitectura definida en el SAD, y realiza el análisis de complejidad ciclomática del componente de mayor riesgo del sistema: el Algoritmo de Clasificación Manchester.

Este artefacto está alineado con:

- SRS IEEE 830: requisitos funcionales RF-001 a RF-010 y no funcionales RNF-001 a RNF-010.
- SAD: arquitectura MVC con Spring Boot (backend), HTML/CSS/JS (frontend) y PostgreSQL (base de datos).
- Manual de Mantenimiento: deuda técnica DEU-005 (ausencia de reporte de cobertura JaCoCo).
- Plan de Prueba: 21 casos de prueba (TC-001 a TC-021), con énfasis en TC-010 y TC-011 para el algoritmo Manchester.

2. Análisis Estático con SonarQube

2.1 Descripción de la herramienta

SonarQube es la plataforma estándar de análisis estático de código en el ecosistema Java/Spring Boot. Examina el código fuente sin ejecutarlo, identificando bugs, vulnerabilidades de seguridad (Security Hotspots), code smells y problemas de cobertura de pruebas. Para MedTriage se proyecta su uso integrado al pipeline CI/CD mediante un análisis automático en cada push a la rama develop.

2.2 Configuración del proyecto en SonarQube

Parámetro	Valor configurado
Nombre del proyecto	MedTriage – Sistema de Triage Médico
Clave del proyecto	medtriage-urgencias-vdr
Versión analizada	1.0 (Sprint 1 – baseline)

Lenguaje principal	Java 17 (backend Spring Boot)
Lenguajes secundarios	JavaScript (frontend), SQL (esquema PostgreSQL)
Rama analizada	develop (integración continua)
Herramienta de cobertura	JaCoCo (integrado en Maven/Gradle)
Perfil de calidad	Sonar way (Java) – configuración estándar
Servidor SonarQube	Instancia local del servidor hospitalario (staging)

2.3 Quality Gate definido — MedTriage Gate

Se define un Quality Gate personalizado denominado MedTriage Gate que el proyecto debe superar en cada análisis para que el código sea candidato a despliegue en producción. Si alguna condición falla, el pipeline se detiene y el equipo responsable debe corregir antes de continuar.

Métrica	Condición	Umbral	Justificación
Cobertura de código (nuevas líneas)	$\geq$	80 %	DEU-005 del Manual de Mantenimiento establece objetivo del 80 % en clases de servicio.
Duplicación de código	$<$	3 %	Código duplicado incrementa la deuda técnica y dificulta el mantenimiento del algoritmo Manchester.
Bugs críticos y bloqueantes	$=$	0	Un bug en el controlador de triaje puede generar clasificaciones erróneas con impacto clínico directo.
Vulnerabilidades de seguridad	$=$	0	El sistema maneja datos clínicos sensibles bajo Ley 1581 de 2012. Cero tolerancia.
Security Hotspots revisados	$=$	100 %	Todos los puntos de calor de seguridad deben ser revisados y marcados como Acknowledged o Fixed.
Code Smells (deuda técnica)	$<$	2 h	Controla la acumulación de deuda técnica por sprint conforme al registro DEU del mantenimiento.
Complejidad ciclomática por método	$\leq$	10	Umbral estándar SonarQube. El Algoritmo Manchester requiere refactorización especial (ver Sección 3).
Líneas de código sin comentar (métodos públicos)	$\geq$	60 %	Garantiza que la API REST documentada con Swagger/OpenAPI tenga respaldo en el código.

2.4 Resultado proyectado del análisis estático (Sprint 1 – Baseline)

Con base en la arquitectura MVC definida en el SAD y las historias de usuario del Sprint 1 (PB-01: Autenticación, PB-02: Registro demográfico, PB-03: Signos vitales), se proyectan los siguientes resultados del primer análisis SonarQube:

Módulo / Componente	Bugs estimados	Vulnerabilidades	Code Smells	Estado QG
Controlador de Autenticación (Spring Security)	0	1 (Hotspot JWT)	2	⚠ Revisar
Controlador de Pacientes (CRUD demográfico)	0	0	1	✓ Pasa
Controlador de Signos Vitales	0	0	1	✓ Pasa
Servicio Manchester (Algoritmo – Sprint 2)	—	—	—	Pendiente Sprint 2
Frontend Login y Registro (JS)	1 (validación)	0	3	⚠ Revisar
Esquema SQL PostgreSQL	0	0	0	✓ Pasa

Nota: el resultado del Controlador de Autenticación requiere revisión del Security Hotspot asociado al manejo del token JWT. Spring Security genera este hotspot por defecto y debe marcarse como 'Acknowledged' tras verificar la implementación correcta de expiración y firma del token.

3. Métricas de Calidad

Densidad de Defectos · Complejidad Ciclomática · Cobertura

3.1 Complejidad Ciclomática — Concepto y fórmula

La complejidad ciclomática $V(G)$ de McCabe mide el número de caminos linealmente independientes a través del código fuente. Se calcula con la fórmula:

$$V(G) = E - N + 2P$$

E = aristas del grafo de flujo | N = nodos | P = componentes conexos (1 para un método)

Equivalente práctico: $V(G)$ = número de puntos de decisión (IF, ELSE IF, WHILE, FOR, CASE, &&, ||, ?) + 1. SonarQube calcula esta métrica automáticamente por método. El umbral recomendado es $V(G) \leq 10$; valores superiores a 15 se consideran de alta complejidad y riesgo.

3.2 Análisis de complejidad: Algoritmo de Clasificación Manchester

El Servicio de Triage Manchester (RF-005, PB-05) es el componente de mayor riesgo del sistema MedTriage. Procesa hasta 52 discriminadores clínicos distribuidos en múltiples motivos de consulta y evalúa 5 niveles de urgencia. A continuación se proyecta la estructura de decisión del método principal `clasificarNivel()`:

3.2.1 Tabla de puntos de decisión proyectados

Bloque lógico	Tipo de decisión	Cantidad estimada	V(G) parcial	Riesgo
Selección de motivo de consulta (22 motivos MTS)	SWITCH / IF-ELSE	22	23	Alto
Evaluación de signos vitales críticos (FC, FR, SpO2, PA, T°)	IF con operadores &&/	12	13	Medio
Evaluación de glucometría fuera de rango	IF simple	3	4	Bajo
Discriminadores Nivel 1 (Rojo – Inmediato)	IF anidado	8	9	Medio
Discriminadores Nivel 2 (Naranja – Muy Urgente)	IF anidado	10	11	Alto
Discriminadores Nivel 3 (Amarillo – Urgente)	IF anidado	10	11	Alto
Discriminadores Nivel 4-5 (Verde/Azul)	IF simple	8	9	Medio
Escala de dolor: umbrales 1-3 / 4-6 / 7-10	IF-ELSE encadenado	3	4	Bajo
Método completo sin refactorización	—	76	77	CRÍTICO

El método clasificarNivel() sin refactorización alcanza $V(G) = 77$, lo que excede ampliamente el umbral SonarQube de 10 y clasifica como código de riesgo CRÍTICO. Este resultado respalda la deuda técnica DEU-002 del Manual de Mantenimiento (desacoplamiento del Algoritmo Manchester como microservicio).

3.2.2 Estrategia de refactorización propuesta

Para reducir la complejidad ciclomática del Algoritmo Manchester por debajo del umbral de 10 por método, se propone la siguiente descomposición:

Método refactorizado	Responsabilidad	V(G) proyectado	Líneas est.	Estado QG
evaluarSignosVitalesCriticos()	Analiza FC, FR, SpO2, PA, temperatura y glucometría.	9	~40	✓ Pasa
clasificarPorMotivoConsulta()	Enruta al evaluador específico según el motivo de consulta seleccionado.	7	~30	✓ Pasa
evaluarDiscriminadoresNivel1()	Evalúa discriminadores de	8	~35	✓ Pasa

	Nivel 1 (Rojo – Inmediato).			
evaluarDiscriminadoresNivel2_3()	Evalúa discriminadores de Niveles 2 y 3.	9	~45	✓ Pasa
evaluarDiscriminadoresNivel4_5()	Evalúa discriminadores de Niveles 4 y 5.	7	~30	✓ Pasa
calcularAporteDolor()	Ajusta nivel según escala de dolor (1-3 / 4-6 / 7-10).	3	~15	✓ Pasa
clasificarNivel() [orquestador]	Llama a los métodos anteriores y determina nivel final.	6	~25	✓ Pasa

3.3 Densidad de Defectos

La densidad de defectos es una métrica que relaciona el número de defectos encontrados con el tamaño del código, expresado en líneas de código (LOC) o en puntos de función. Su fórmula es:

$$\text{Densidad de Defectos} = \text{Número de Defectos} / \text{KLOC}$$

(KLOC = miles de líneas de código)

3.3.1 Estimación de tamaño del proyecto MedTriage

Componente	Líneas de Código Estimadas (LOC)
Backend Spring Boot (Controladores + Servicios + Repos)	~2 800 LOC
Algoritmo Manchester (Servicio de Triage)	~600 LOC (refactorizado en 7 métodos)
Frontend HTML/CSS/JavaScript	~1 400 LOC
Scripts SQL (esquema + datos semilla)	~300 LOC
Tests unitarios e integración (JUnit + Jest)	~1 200 LOC
TOTAL ESTIMADO (sin tests)	~5 100 LOC (~5,1 KLOC)

3.3.2 Proyección de densidad de defectos por sprint

Sprint	Historias incluidas	LOC estimado	Defectos objetivo	Densidad objetivo
Sprint 1	PB-01, PB-02, PB-03	~1 200 LOC	≤ 3	≤ 2,5 defectos/KLOC

Sprint 2	PB-04, PB-05, PB-06	~1 500 LOC	≤ 4	≤ 2,7 defectos/KLOC
Sprint 3	PB-07, PB-08, PB-09	~1 200 LOC	≤ 3	≤ 2,5 defectos/KLOC
Sprint 4	PB-10 al PB-14 (RNF)	~1 200 LOC	≤ 3	≤ 2,5 defectos/KLOC
TOTAL PROYECTO	—	~5 100 LOC	≤ 13	≤ 2,5 defectos/KLOC

El umbral de referencia para sistemas de software médico-crítico es 3 defectos/KLOC (NASA Software Engineering Laboratory). MedTriage adopta el objetivo más exigente de ≤ 2,5 defectos/KLOC dado el impacto clínico de una clasificación incorrecta del triaje.

3.4 Cobertura de Código — Configuración JaCoCo

JaCoCo (Java Code Coverage) es la herramienta definida en el Plan de Prueba para medir la cobertura de código del backend Spring Boot. Su integración con SonarQube permite visualizar los resultados en el dashboard del proyecto.

Parámetro JaCoCo	Valor configurado
Herramienta	JaCoCo 0.8.x integrado en Maven (plugin jacoco-maven-plugin)
Informe generado	target/site/jacoco/index.html (local) + exportado a SonarQube
Cobertura mínima requerida (clases de servicio)	80 % (DEU-005 del Manual de Mantenimiento)
Clases excluidas del análisis	Entidades JPA (@Entity), DTOs, clases de configuración Spring
Clases prioritarias de cobertura	TriajeService, ColaPriorizadaService, AutenticacionService
Tipo de cobertura medida	Cobertura de línea + cobertura de rama (branch coverage)
Cobertura objetivo del Algoritmo Manchester	≥ 90 % de rama (por impacto clínico directo)

4. Dashboard de Calidad Projectado — Vista Ejecutiva

El siguiente panel resume el estado proyectado de calidad del sistema MedTriage al finalizar el desarrollo completo (Sprint 4), siempre que se implementen las estrategias de refactorización y pruebas definidas en este reporte.

Indicador de Calidad	Valor proyectado	Umbral definido	Estado
Quality Gate SonarQube	PASSED	PASSED	✓
Bugs críticos/bloqueantes	0	= 0	✓

Vulnerabilidades de seguridad	0	= 0	✓
Cobertura de código (backend)	≥ 80 %	≥ 80 %	✓
Cobertura Algoritmo Manchester	≥ 90 %	≥ 90 %	✓
Duplicación de código	< 2 %	< 3 %	✓
Complejidad ciclomática máx. por método	V(G) = 9	≤ 10	✓
Densidad de defectos total	≤ 2,5 / KLOC	≤ 2,5 / KLOC	✓
Deuda técnica total (SonarQube)	< 2 h	< 2 h	✓
Security Hotspots revisados	100 %	100 %	✓

5. Plan de Acción para Alcanzar el Quality Gate

ID	Acción	Descripción	Responsable	Sprint
QA-01	Configurar SonarQube en pipeline CI	Integrar sonar-scanner en el pipeline de desarrollo. Configurar token de autenticación y URL del servidor.	DevOps	Sprint 1
QA-02	Configurar JaCoCo en el pom.xml	Añadir jacoco-maven-plugin y configurar exportación a SonarQube como cobertura del proyecto.	Backend	Sprint 1
QA-03	Refactorizar clasificarNivel()	Descomponer el método en 7 submétodos según la estrategia de la Sección 3.2.2 para reducir $V(G) \leq 10$.	Backend	Sprint 2
QA-04	Pruebas unitarias del Algoritmo Manchester	Implementar casos de prueba TC-010 y TC-011 con JUnit para lograr cobertura de rama ≥ 90 % en TriajeService.	Backend / QA	Sprint 2
QA-05	Revisar Security Hotspot JWT	Marcar el hotspot de Spring Security como Acknowledged tras verificar firma y expiración del token JWT.	Backend	Sprint 1
QA-06	Corregir bug validación Frontend	Resolver el bug de validación identificado en el formulario de login (JavaScript) antes del cierre del Sprint 1.	Frontend	Sprint 1
QA-07	Generar reporte final JaCoCo	Al finalizar Sprint 4, generar el reporte de cobertura definitivo y adjuntarlo como	QA	Sprint 4

		artefacto de cierre del proyecto.		
--	--	-----------------------------------	--	--